PrismDM: An Accurate and Efficient Framework for Privacy-Preserving Diffusion Model Sampling

Anonymous Author(s)

ACM Reference Format:
Anonymous Author(s). 2018. PrismDM: An Accurate and Efficient Framework for Privacy-Preserving Diffusion Model Sampling. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

A Detailed Model Structure and Bias Amplification Analysis

In this section, we analyze the distribution of Element-wise Operators (EWO) and Vector-wise Operators (VWO) across the different layers of the DDPM/DDIM models. We use the model's architecture to examine where EWO and VWO are applied and why EWO are more prevalent in DMs.

Model Architecture Overview.

The DDPM/DDIM models are based on a UNet architecture, composed of multiple layers:

- Downsampling layers progressively reduce the spatial resolution while increasing feature channels.
- Residual blocks are used within both downsampling and upsampling layers to perform feature extraction.
- Middle layers serve as the bottleneck, processing abstract features.
- Upsampling layers restore the image resolution and utilize skip connections to merge high-level and low-level features.

EWO (Element-wise Operators).

EWO operations like ReLU, SiLU, and Mish are applied element-wise across the feature maps. These operations are used extensively in the residual blocks of both the downsampling and upsampling layers. They are also present in the middle layers, where they help in learning complex, non-linear transformations.

- EWO is applied primarily after convolution operations, where each element of the feature map is processed individually.
- These operators are key for learning local features, which are critical for the model's ability to generate high-quality images.

VWO (Vector-wise Operators).

VWO operations like Softmax aggregate information across the entire feature map and capture global dependencies. These operations are more computationally intensive and are used selectively in the network.

- VWO operations are applied mainly in the middle layers, where capturing global context is important for generating coherent and consistent output.

Layer-wise Distribution of EWO and VWO.

- Downsampling layers: Primarily use EWO (e.g., ReLU, SiLU) in the residual blocks, as these layers focus on local feature extraction.
- Middle layers: Both EWO and VWO are present. However, EWO still dominates, as the majority of transformations in these layers involve element-wise operations.
- Upsampling layers: Like the downsampling layers, these layers mainly rely on EWO for feature transformation and refinement, with fewer instances of VWO.

Thus, EWO is much more prevalent throughout the model, appearing in almost every layer, whereas VWO is primarily limited to the middle layers.

Bias Amplification Analysis.

The bias amplification analysis in the main paper revealed iterative error magnification in DM sampling, which guided the design of our acceleration and Feature Alignment schemes. These insights can also motivate future adaptive methods with step-dependent acceleration.

B Nonlinear Operators Accelerations and Secure MPC Protocols

While the multi-objective optimization framework we propose offers a general and flexible approach for accelerating nonlinear operators in MPC-based secure DM sampling, directly applying it in full generality may incur nontrivial tuning cost. In practice, we introduce a set of simplifications guided by both operator characteristics and MPC-specific computational constraints to make the optimization efficient and stable.

Practical Instantiation of the Searching Procedure.

The optimization objective in Eq. (5) in the main paper defines a general multi-objective formulation over interval boundaries $\{z_i\}$, polynomial degrees $\{L_i\}$, and sparsity patterns $\{j_i\}$. In principle, this objective can be solved by generic multi-objective search algorithms. However, directly optimizing over the full combinatorial space is impractical in our setting due to the exponential growth of candidates and the high evaluation cost under MPC.

In practice, we instantiate the searching process via a Bayesian Optimization approach, which provides an efficient and probabilistic method for searching over high-dimensional spaces. Bayesian Optimization iteratively builds a surrogate model of the objective

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

function, typically using Gaussian Processes, and uses an acquisition function to balance exploration and exploitation of the parameter space.

First, we construct a finite candidate set for interval boundaries $\{z_i\}$ using empirical input distributions collected from pretrained diffusion models. For each nonlinear operator, we identify regions where the operator exhibits near-constant or near-linear behavior, and fix boundary candidates accordingly. This directly reduces the dimensionality of the search space and eliminates unnecessary boundary configurations.

Second, for each interval, we enumerate a set of polynomial degrees $L_i \in \{1, 2, 3\}$ and corresponding sparsity patterns j_i, restricted to monomials that are efficient under ABY3. In particular, monomials that can reuse intermediate results (e.g., x^2, $(x^2)^2$) are favored, while higher-order or chained multiplications are excluded from the candidate set. This step explicitly embeds MPC cost awareness into the search space construction.

Third, for each candidate configuration $(\{z_i\}, \{L_i\}, \{j_i\})$, we evaluate the objective in Eq. (5) by estimating the approximation error $\mathcal{E}_{\text{app}}$, the continuity penalty $\mathcal{E}_{\text{cont}}$, and the inference cost τ. The cost term τ is instantiated using the average secure inference latency measured under the target MPC protocol, as described earlier.

Finally, we select the configuration with the minimum objective value under an acceptable quality constraint. In practice, we observe that small values of Λ lead to discontinuous approximations and visible artifacts in generated samples. Therefore, we fix $\Lambda = 100$ and perform the search under this setting. The balancing coefficients α and β are chosen implicitly by selecting the minimum-cost configuration that satisfies the target sample quality threshold, rather than by explicitly tuning their absolute values.

This structured search procedure preserves the spirit of the proposed multi-objective optimization framework while remaining computationally feasible and stable in MPC-based diffusion model sampling.

Domain Simplification Based on Operator Features.

We first leverage the intrinsic behavior of common nonlinear operators to prune unnecessary approximation complexity at the interval level:

- For **SiLU** and **Mish**, their outputs on the far-left and far-right tails are close to constant or linear:
  - In the region $x < -6$, both functions are nearly zero, and thus approximated by 0;
  - In the region $x > 6$, both functions approach identity, and are approximated by x;

  Hence, we only fit piecewise polynomials in the central region $[-6, 6]$, which is further divided into two intervals with different degrees.
- For e^x in Softmax, we utilize the fact that:
  - Under the 18-bit MPC protocol (i.e., in the ring $\mathbb{Z}_{2^{18}}$), values smaller than e^{-14} are indistinguishable from zero;
  - Therefore, we clip any $x < -14$ to 0 and perform polynomial fitting only in the domain $[-14, 0]$.

Efficiency-Aware Polynomial Design.

In secure computation protocols such as ABY3, arithmetic operations are not uniformly costly. Notably:

- Power operations like x^4 can be more efficient than chained multiplications, due to reuse of intermediate computations, e.g., $(x^2)^2$;
- The computational cost of a polynomial depends not only on the number of monomials, but also on how terms are constructed.

To reflect this, we benchmarked candidate polynomial forms by measuring their actual runtime under our ABY3-based MPC engine. These empirical latency values are integrated into our optimization cost C in the objective function. Polynomial configurations with lower-cost monomials (e.g., x^2 or $(x^2)^2$) are favored, while expensive compositions are penalized.

Final Configuration and Results.

After empirical tuning, we select the regularization weights $\Lambda = 100$ for both EWOs and VWOs as the final configuration. The resulting piecewise accelerations are listed below with detailed parameters in Table 1.

$$\text{EWO}(x) = \begin{cases} 0 & x < -6 \\ a_1 x + a_0 & -6 \le x < z \\ b_3 x^3 + b_2 x^2 + b_1 x + b_0 & z \le x < 6 \\ x & x \ge 6 \end{cases} \tag{1}$$

$$e^x = \begin{cases} 0 & x < -14 \\ a_3 x^3 + a_2 x^2 + a_1 x + a_0 & -14 \le x < -3.07 \\ b_3 x^3 + b_2 x^2 + b_1 x + b_0 & -3.07 \le x < 0 \end{cases} \tag{2}$$

EWO	**Coefficients**			
SiLU	a_1	a_0	z	
	-0.05722988	-0.32197971	-1.86	
	b_3	b_2	b_1	b_0
	0.04436355	0.54414428	0.16303972	-0.01863412
Mish	a_1	a_0	z	
	-0.05448525	-0.30983219	-2.22	
	b_3	b_2	b_1	b_0
	0.09984283	0.60460080	0.17227525	-0.01895927
e^x	a_3	a_2	a_1	a_0
	0.00014351	0.00426741	0.04082211	0.12514002
	b_3	b_2	b_1	b_0
	0.04096367	0.31576001	0.89263596	0.98513184

Table 1: Approximated coefficients for Accuracy-Aware Accelerations

This simplification strategy retains high approximation fidelity while significantly reducing fitting complexity and inference cost, aligning well with efficiency constraints in privacy-preserving inference under MPC protocols.

C Training Details

DDIM / DDPM Pre-Training. As **CipherDM** only provides a single pre-trained model without any training details (e.g., nonlinear operator type, number of training epochs, optimizer settings), we pre-train all vanilla DDIM and DDPM models used in our evaluations to ensure fairness and reproducibility.

For DDIM models evaluated on **MNIST-01**, we train from scratch using following settings, with "Adam" denotes the Adam optimizer [3]

- **Epochs:** 4000
- **Batch size:** 128
- **Learning rate:** 2×10^{-4}
- **Optimizer:** Adam
- **Random seed:** 42

FID scores in Table 1 of the main paper are computed using 12,000 generated images.

For DDIM and DDPM models evaluated on **CIFAR-10**, we adopt:

- **Epochs:** 10,000
- **Batch size:** 128
- **Learning rate:** 2×10^{-4}
- **Optimizer:** Adam
- **Random seed:** 42

All aligned models in PrismDM are fine-tuned from their corresponding pre-trained vanilla models using the proposed feature alignment algorithm.

Fine-Tuning for DDPM/DDIM Models. For all DDIM/DDPM models (on both MNIST-01 and CIFAR-10), the fine-tuning configuration is:

- **Epochs:** 50
- **Batch size:** 128
- **Learning rate:** 2×10^{-4}
- **Optimizer:** Adam
- **Random seed:** 42

Fine-Tuning for StableDiffusion Models. We fine-tune the StableDiffusion V1-4 on two datasets: Naruto (lambdalabs/blip-caption) and COCO-2017.

- **Epochs:** 10
- **Batch size:** 1
- **Learning rate:** 2×10^{-4}
- **Optimizer:** Adam
- **Random seed:** 42

For COCO-2017, we randomly shuffle the dataset and select the first 30,000 samples to control training cost. The text encoder is kept unchanged, and no LoRA, EMA, or freezing strategies are applied. All results are generated with a fixed seed to ensure deterministic sampling and reproducibility.

Implementation. All models are implemented using PyTorch 2.4.0 and integrated with the SecretFlow backend for secure inference. All training and evaluation pipelines follow standard reproducibility checklists. No data augmentation or dropout is applied unless otherwise noted.

Hyperparameter Selection. The training hyperparameters $\lambda_{\text{EWO}}, \lambda_{\text{VWO}}, \lambda_{\text{FA}}$ are tuned via a hyperparameter sweep based on the quality of generated images. The impact of these hyperparameters is analyzed in the ablation study (Section 4.4 of the main paper).

D FID Thresholds for DM Practicality Evaluation

To determine whether a generated image is of acceptable quality, we establish dataset-specific FID thresholds based on visual assessment and human perception. Specifically, we conducted a user study where evaluators reviewed generated samples across varying FID scores and determined the minimum quality level that remained visually coherent and semantically meaningful. The thresholds are shown in Table 2.

Dataset	Acceptable FID Threshold
MNIST-01	≤ 80
CIFAR-10	≤ 80
Naruto	≤ 150
COCO	≤ 40

Table 2: FID thresholds for judging DM practicality.

These values reflect the point at which generated samples were judged by human annotators to be sufficiently clear and structurally consistent. Notably, while FID is widely used for evaluating generative models, recent works have emphasized that perceptual quality does not always correlate linearly with FID scores, especially across datasets of different complexity.

Our thresholds are supported by consensus and observations from prior works [2, 4]. These works demonstrate that thresholds must be calibrated with respect to both statistical distribution metrics and human-perceived realism. Our thresholds serve as a practical balance between empirical FID measurements and subjective visual judgment.

E Cryptographic Notations and Underlying Protocols

MPC Framework. We adopt secure Multi-Party Computation (MPC) as the core cryptographic scheme in this work. Specifically, MPC harmonizes privacy, efficiency, and accuracy while avoiding drawbacks of other cryptographic-based methods, such as the precision degradation in differential privacy [1].

The basic principle of MPC is secret sharing, where sensitive data is transformed into secret shares and distributed among separate non-colluding computation parties. These parties then collaboratively execute MPC protocols, known as computation primitives, to process all data and intermediate results in the form of secret shares, thereby preserving privacy. Additionally, since the 3-party computation (3PC) framework exhibits superior efficiency compared to 2PC framework [7], we adopt the widely used 3PC framework ABY3 [6] and its 2-out-of-3 secret sharing scheme.

2-out-of-3 Secret Sharing. In 2-out-of-3 secret sharing, a secret x in ring $\mathbb{Z}_{2^l}$ is divided into shares $[\![x]\!] = x_0 + x_1 + x_2$, where l is the bit length and is set to 18 in this work. These shares are distributed among three computation parties P_i, where each P_i holds $(x_i, x_{(i+1) \bmod 3})$. We summarize the basic computation primitives used to form composite protocols (e.g., full inference protocol) below.

- **Addition.** For secrets $[\![x]\!]$, $[\![y]\!]$ and public constants c_1, c_2, c_3, linear additions are computed as:

$$[\![c_1x + c_2y + c_3]\!] = c_1[\![x]\!] + c_2[\![y]\!] + c_3. \tag{3}$$

  Each party P_i locally computes $c_1x_i + c_2y_i + \delta_{i0}c_3$, preserving additive homomorphism, where δ_{i0} is 1 for $i = 0$ and 0 otherwise.
- **Multiplication.** For secrets $[\![x]\!]$, $[\![y]\!]$, the product $[\![xy]\!]$ expands as:

$$[\![xy]\!] = \sum_{0\le i,j\le 2} x_iy_j = w_0 + w_1 + w_2, \tag{4}$$

  Each party P_i locally computes $w_i = x_iy_i + x_{i+1}y_i + x_iy_{i+1}$ and sends masked $w_i' = w_i - \alpha_i$ to P_{i+1}. Masks α_i, which satisfy $\sum_i \alpha_i = 0$, are generated and shared during the setup phase. The product $[\![xy]\!]$ is then formed and shared as $\{(w_0', w_1'), (w_1', w_2'), (w_2', w_0')\}$.
- **Underlying Protocols.** PrismDM leverages SOTA computation primitives (e.g., comparison, truncation) implemented in SecretFlow [5], treating them as black-box components.

Basic cryptographic notations used in PrismDM's MPC protocols are listed in Table 3.

Notations	Meanings
$\mathbb{Z}_{2^\ell}$	discrete ring modulo 2^ℓ
P_i	computation party i in 3PC
$[\![\cdot]\!]$	Arithmetic Sharing in $\mathbb{Z}_{2^\ell}$
$[\![\cdot]\!]^B$	Boolean Sharing in $\mathbb{Z}_2$
$\mathbf{x}$	uppercase bold letter denotes vector
x	lowercase letter denotes scalar
x^i	the power of scalar x with an order of i
α_i	random value pre - computed by P_i
$\oplus$	the operation of XOR

Table 3: Cryptographic Notation table.

PrismDM leverages state-of-the-art MPC primitives (e.g., comparison, truncation) implemented in SecretFlow, treated them as black-box components. Input/output specifications for these protocols are detailed in Table 4.

Protocol	Semantic	Expression
Π_{LT}	$z = 1\{x < y\}$	Less Than
Π_{Max}	$z = \text{maximum}(\mathbf{x})$	Maximum
Π_{Recip}	$z = 1/x$	Reciprocal
Π_{Square}	$z = x^2$	Square
Π_{MulBA}	$z = b \cdot x$	Boolean-Arithmetic Multiplication

Table 4: Underlying black-box protocols.

The final MPC protocols for secure SiLU and Mish evaluation are presented in Algorithm 1, respectively.

The final MPC protocol for secure Softmax evaluation is presented in Algorithm 2.

Algorithm 1 Secure SiLU/Mish Protocol $\Pi_{\text{SiLU/Mish}}$

Input: P_i holds the 2-out-of-3 replicated secret share $[\![x]\!]$ for $i \in \{0,1,2\}$, segment point z, fitted polynomial coefficients $\{A_0[j_0]\}$ for $j_0 \in \{0,1,2,3\}$, and $\{A_1[j_1]\}$ for $j_1 \in \{0,1\}$.

Output: P_i obtains the 2-out-of-3 replicated secret share $[\![y]\!]$ for $i \in \{0,1,2\}$, where $y = \text{SiLU}(x)/\text{Mish}(x)$.

1: P_0, P_1, P_2 jointly compute $[\![g_0]\!]^B$, $[\![g_1]\!]^B$, $[\![g_2]\!]^B$, $[\![h_0]\!]^B$, $[\![h_1]\!]^B$, and $[\![h_2]\!]^B$, where

$$[\![g_0]\!]^B = \Pi_{\text{LT}}([\![x]\!], -6), \quad [\![g_1]\!]^B = \Pi_{\text{LT}}([\![x]\!], z),$$
$$[\![g_2]\!]^B = \Pi_{\text{LT}}(6, [\![x]\!]), \quad [\![h_0]\!]^B = [\![g_0]\!]^B \oplus [\![g_1]\!]^B,$$
$$[\![h_1]\!]^B = [\![g_1]\!]^B \oplus [\![g_2]\!]^B \oplus 1, \quad [\![h_2]\!]^B = [\![g_2]\!]^B.$$

2: **for** $j = 2, 3$ **do**
3: P_0, P_1, P_2 jointly compute $[\![x^j]\!] = \Pi_{\text{Mul}}([\![x^{j-1}]\!], [\![x]\!])$
4: **end for**
5: P_0, P_1, P_2 locally compute

$$[\![F_0]\!] = \sum_{j_0\in\{0,1,2,3\}} [\![x^{j_0}]\!] \cdot A_0[j_0], \quad [\![F_1]\!] = \sum_{j_1\in\{0,1\}} [\![x^{j_1}]\!] \cdot A_1[j_1].$$

6: **return** $[\![y]\!] = \Pi_{\text{Mul}_{\text{BA}}}([\![h_0]\!]^B, [\![F_0]\!]) + \Pi_{\text{Mul}_{\text{BA}}}([\![h_1]\!]^B, [\![F_1]\!]) + \Pi_{\text{Mul}_{\text{BA}}}([\![h_2]\!]^B, [\![x]\!])$.

Algorithm 2 Secure Softmax Protocol Π_{Softmax}

Input: P_i holds the 2-out-of-3 replicated secret share $[\![\mathbf{x}]\!]$ for $i \in \{0,1,2\}$, where $\mathbf{x}$ is a vector of size n, segment point z, and fitted polynomial coefficients $\{A_0[j_0]\}$ for $j_0 \in \{0,1,2\}$ and $\{A_1[j_1]\}$ for $j_1 \in \{0,1,2\}$.

Output: P_i obtains the 2-out-of-3 replicated secret share $[\![\mathbf{y}]\!]$ for $i \in \{0,1,2\}$, where $\mathbf{y} = \text{Softmax}(\mathbf{x})$.

1: P_0, P_1, P_2 jointly compute the maximum $[\![\bar{x}]\!] = \Pi_{\text{Max}}([\![\mathbf{x}]\!])$, and then locally compute the shifted input $[\![\hat{\mathbf{x}}]\!] = [\![\mathbf{x}]\!] - [\![\bar{x}]\!] - \epsilon$.
2: P_0, P_1, P_2 jointly compute Boolean shares $[\![\mathbf{g}_1]\!]^B = \Pi_{\text{LT}}(z, [\![\hat{\mathbf{x}}]\!])$.
3: **for** $j = 2, 3$ **do**
4: P_0, P_1, P_2 jointly compute $[\![\hat{\mathbf{x}}^j]\!] = \Pi_{\text{Mul}}([\![\hat{\mathbf{x}}^{j-1}]\!], [\![\hat{\mathbf{x}}]\!])$.
5: **end for**
6: P_0, P_1, P_2 locally compute

$$[\![\mathbf{F}_0]\!] = \sum_{j_0\in\{0,1,2\}} [\![\hat{\mathbf{x}}^{j_0}]\!] \cdot A_0[j_0], \quad [\![\mathbf{F}_1]\!] = \sum_{j_1\in\{0,1,2\}} [\![\hat{\mathbf{x}}^{j_1}]\!] \cdot A_1[j_1].$$

7: P_0, P_1, P_2 locally compute

$$[\![\mathbf{u}]\!] = \Pi_{\text{Mul}_{\text{BA}}}([\![\mathbf{g}_1]\!]^B, [\![\mathbf{F}_0]\!]) + \Pi_{\text{Mul}_{\text{BA}}}(1 - [\![\mathbf{g}_1]\!]^B, [\![\mathbf{F}_1]\!]).$$

8: P_0, P_1, P_2 jointly compute $[\![s]\!] = \sum_{l=1}^{n} [\![\mathbf{u}[l]]\!]$ and $[\![r]\!] = \Pi_{\text{Recip}}([\![s]\!])$.
9: **return** $[\![\mathbf{y}]\!] = \Pi_{\text{Mul}}([\![\mathbf{u}]\!], [\![r]\!])$.

F Acceleration Strategies Ablation

To isolate the contribution of the proposed acceleration strategy in PrismDM, we conduct an ablation study comparing multiple acceleration schemes under the same training algorithm. Specifically, all models are fine-tuned using our feature alignment algorithm in the main paper, ensuring that the observed performance differences stem solely from the acceleration schemes.

We compare our acceleration scheme (noted as "PrismDM") with three representative strategies:

- **MPCFormer:** A fixed acceleration method used by MPCFormer, where the SiLU and Mish activations are replaced with a single handcrafted polynomial $0.125x^2 + 0.25x + 0.5$. For Softmax, the function is approximated using a normalized square form: $\frac{(x+5)^2}{\sum(x+5)^2}$. No segmentation or distribution-aware fitting is applied.
- **Quadratic without segmentation:** For SiLU and Mish, a second-degree polynomial is fitted on the interval $[-6, 6]$ using least squares methods. For Softmax, its core component, the exponential function e^x uses the high-precision acceleration used in PrismDM. This baseline reflects a simplified but distribution-aware design for EWO functions.
- **CipherDM:** This method faithfully follows CipherDM's design. It fits e^x for Softmax using a 7-th degree polynomial over $[-14, 0]$ after truncation. For SiLU and Mish, two polynomial pieces are used: a 2nd-degree polynomial on $[-6, -2]$ and a 6th-degree polynomial on $[-2, 6]$. All polynomials are manually selected without joint optimization.

All models are evaluated on the COCO dataset using the same StableDiffusion backbone and training pipeline. The FID scores are computed from 12,000 generated images and shown in Table 5.

Acceleration Scheme	FID (↓)
MPCFormer	47.34
Quadratic without segmentation	37.58
CipherDM	31.68
PrismDM	**28.59**

Table 5: FID scores on COCO dataset using different acceleration strategies, with identical feature alignment training.

These results clearly demonstrate that PrismDM's acceleration method achieves the highest sample quality. Notably, even the naive quadratic baseline benefits significantly from being distribution-aware and from using PrismDM's Softmax acceleration. Meanwhile, MPCFormer's hardcoded approximations are too coarse to retain detail under secure sampling. CipherDM performs better with its manually segmented polynomials, but still lacks the adaptive bias-cost balancing achieved by PrismDM.

G Fine-Tuning Methods Ablation and PrismDM's Advantages

While MPCFormer also employs a fine-tuning strategy for improving surrogate model accuracy under secure computation, there exists a critical difference between our approach and theirs in the design of the training objective.

Loss Function Design. MPCFormer adopts a standard mean squared error (MSE) loss for training the accelerated model. This generic objective lacks operator-specific sensitivity and may fail to effectively regularize structured distortions introduced by low-precision approximations.

In contrast, our proposed feature alignment algorithm explicitly distinguishes between different types of nonlinear operators and applies tailored loss functions accordingly:

- For **element-wise operators** (EWO) such as ReLU, SiLU, and Mish, we employ the **L2 norm** to measure value-level deviation.
- For **vector-wise operators** (VWO) such as Softmax, we utilize the **KL divergence** to better capture distributional differences.

This targeted alignment strategy is particularly beneficial when using aggressive (low-degree) polynomial approximations, where naive MSE loss fails to sufficiently constrain the acceleration bias.

Empirical Comparison. To quantify this effect, we conduct experiments on the COCO dataset using StableDiffusion with approximated nonlinear operators.

Fine-tuning Method	FID (↓)
MPCFormer	214.62
PrismDM	**47.34**

Table 6: Comparison of FID scores on COCO dataset after fine-tuning.

As shown in Table 6, our method achieves a **4.53× improvement** in FID compared to MPCFormer. This demonstrates that merely applying MSE is insufficient to mitigate bias amplification in complex DMs, while our operator-aware loss function leads to significantly better image quality under secure inference constraints.

Based on the ablation results presented in F, it can be concluded that PrismDM outperforms MPCFormer under both acceleration strategies and fine-tuning settings, and consequently achieves better overall performance.

H Image Quality with Low Inference Steps

We list the FIDs of images generated by DDIM model on MNIST-01 dataset under T=50 in the main paper. For smaller T=10, detailed results are listed in Table 7.

As shown in Table 7, our accuracy-aware acceleration scheme and feature alignment scheme significantly improves the quality of generated images, with FID improvements ranging from 1.07× to 7.23× under acceleration conditions. It is worth noting that aligned models with accelerations not only achieve image quality comparable to vanilla models without accelerations, but also surpass their vanilla counterparts in some cases(e.g., ReLU-based models). Meanwhile, for low T=10, acceleration methods have minor impacts on DM based on simple activation (ReLU), but exhibit more negative effects on complex architectures and nonlinear operators. This aligns with our conclusions in the main paper.

I PrismDM's Extension to Other Fields

PrismDM is primarily designed for score-based diffusion models (SDEs) where iterative sampling processes are used to generate samples. However, the core principles and techniques behind PrismDM can be extended to other types of SDEs that exhibit similar iterative sampling dynamics.

Models	DDIM DM w/ T=10		
	ReLU	SiLU	Mish
SecretFlow	59.61	**45.99**	**64.60**
CipherDM	59.75	88.88	395.95
PrismDM w/o FA	58.26	70.62	403.94
PrismDM	**55.49**	53.54	54.76
Accuracy ↑ w/ PDM	1.07×	1.66×	7.23×

Table 7: FIDs under different acceleration methods. FID scores are computed using 12,000 generated images per DM. "w/o FA" denotes models without feature alignment. "Accuracy ↑ w/ PDM" indicates $\text{FID}_{\text{CipherDM}}/\text{FID}_{\text{PrismDM}}$.

Generalization to Other SDEs. Our method leverages polynomial approximations and tailored acceleration strategies to mitigate bias during sampling process. This approach is not limited to score-based models; it can be applied to other SDEs that involve iterative steps, such as those used in molecular simulations or other stochastic differential equations.

Therefore, while PrismDM was evaluated on score-based DMs in this work, the method can be generalized to any SDE model that shares a similar iterative sampling structure.

J Extended Threat Model Discussion

Honest-but-Curious Threat Model. The security analysis of PrismDM is based on the ABY3 MPC protocol under the honest-but-curious (HBC) threat model, where computation parties are assumed to follow the protocol specification while potentially attempting to infer additional information from observed messages. Under this model, ABY3 provides formal privacy guarantees as long as the prescribed corruption threshold is not exceeded. Our security proof explicitly operates within this setting.

Limitations with Respect to Collusion. Like other secret-sharing-based MPC protocols, ABY3 assumes that the number of colluding parties does not exceed its security threshold. If multiple computation parties collude beyond this limit, the privacy guarantees no longer hold, and sensitive information may be reconstructed. This limitation is inherent to the underlying secret-sharing mechanism and is not specific to PrismDM. In practice, this assumption is commonly adopted in MPC deployments where computation parties are operated by independent organizations or executed in isolated administrative domains, which reduces the feasibility of large-scale collusion.

Side-Channel Considerations. Side-channel attacks, such as those exploiting timing variations, memory access patterns, or communication behavior, are a general concern for cryptographic implementations. These attacks target the system and implementation layers and are largely orthogonal to the cryptographic design of secret-sharing-based MPC protocols. As a result, potential side-channel leakage is not specific to PrismDM and does not arise from its acceleration framework. Existing countermeasures developed for MPC systems, including constant-time implementations, traffic shaping, and hardware-assisted protections, can be applied in conjunction with PrismDM to further mitigate such risks.

Beyond the HBC Model. While our current analysis focuses on the HBC setting, PrismDM is not inherently tied to this threat model. Adopting stronger adversarial models, such as malicious security or insider threat resistance, would require integrating MPC protocols with additional verification mechanisms, including consistency checks or verifiable computation. These extensions primarily affect the underlying MPC layer and can be combined with PrismDM without altering its core optimization framework, albeit at increased computational and communication cost.

References

[1] Mohammad Al-Rubaie and J. Morris Chang. 2019. Privacy-Preserving Machine Learning: Threats and Solutions. *IEEE Security & Privacy* 17, 2 (2019), 49–58. doi:10.1109/MSEC.2018.2888775

[2] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. 2017. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/8a1d694707eb0fefe65871369074926d-Paper.pdf

[3] Diederik P. Kingma and Jimmy Ba. 2017. Adam: A Method for Stochastic Optimization. arXiv:1412.6980 [cs.LG] https://arxiv.org/abs/1412.6980

[4] Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen, and Timo Aila. 2019. Improved Precision and Recall Metric for Assessing Generative Models. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/0234c510bc6d908b28c70ff313743079-Paper.pdf

[5] Junming Ma, Yancheng Zheng, Jun Feng, Derun Zhao, Haoqi Wu, Wenjing Fang, Jin Tan, Chaofan Yu, Benyu Zhang, and Lei Wang. 2023. SecretFlow-SPU: A Performant and User-Friendly Framework for Privacy-Preserving Machine Learning. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 17–33. https://www.usenix.org/conference/atc23/presentation/ma

[6] Payman Mohassel and Peter Rindal. 2018. ABY3: A Mixed Protocol Framework for Machine Learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) *(CCS '18)*. Association for Computing Machinery, New York, NY, USA, 35–52. doi:10.1145/3243734.3243760

[7] Payman Mohassel, Mike Rosulek, and Ye Zhang. 2015. Fast and Secure Three-party Computation: The Garbled Circuit Approach. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security* (Denver, Colorado, USA) *(CCS '15)*. Association for Computing Machinery, New York, NY, USA, 591–602. doi:10.1145/2810103.2813705